

Estruturas Lineares: Arrays Dinâmicos e Listas Encadeadas

COMP0497 – Algoritmos e Estrutura de Dados 1

Prof. Dr. André Yoshiaki Kashiwabara

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

1. Distinguir **Tipo Abstrato de Dados** (ADT) de implementação concreta.
2. Explicar o funcionamento e a análise amortizada de **arrays dinâmicos**.
3. Implementar e analisar operações em **listas simplesmente ligadas**.
4. Comparar as quatro **convenções** de organização de listas.
5. Estender o raciocínio para **listas duplamente ligadas**.



Conceitos Fundamentais

Definição

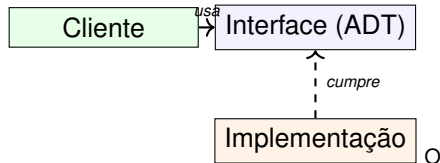
Uma estrutura de dados é a combinação de:

1. Uma **coleção de dados** organizados de forma particular.
 2. As **operações** que podem ser aplicadas a esses dados.
- **Analogia:** Pense em uma biblioteca. Os livros são os dados, mas as estantes e o sistema de catalogação são a **estrutura**.
 - A escolha da estrutura determina a **complexidade** do algoritmo.

A Equação de Wirth

Algoritmos + Estruturas de Dados = Programas

- Um **ADT** define o *quê* (operações), não o *como* (implementação).
- **Independência:** Você pode usar uma Fila sem saber se ela é feita com Array ou Lista Ligada.
- **Exemplos Clássicos:**
 - Pilhas (LIFO)
 - Filas (FIFO)
 - Listas Ordenadas



cliente depende apenas da interface, nunca da implementação.

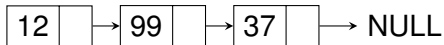
Lineares (foco desta aula)

- **Arrays:** Acesso rápido via índice.
- **Listas Ligadas:** Flexibilidade de inserção/remoção.

Não-Lineares (aulas futuras)

- **Árvores:** Organização hierárquica.
- **Grafos:** Relações arbitrárias.

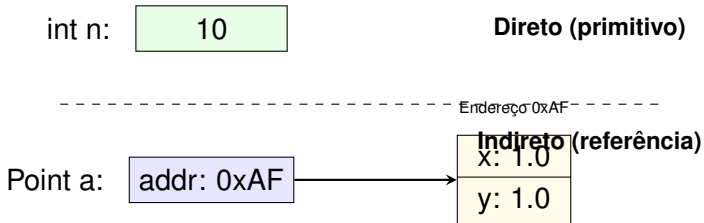
Lista Ligada (Dinâmica)



Array (Contíguo)



índice 0 índice 1 índice 2

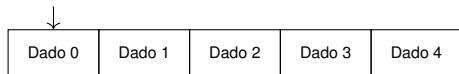


- **Primitivo:** a variável é o valor.
- **Referência:** a variável *aponta* para o objeto.
- Mais barato mover referências que copiar objetos.
- Base para Listas, Árvores e Grafos.

Arrays e Arrays Dinâmicos

- **Definição:** Coleção fixa de elementos do mesmo tipo.
- **Propriedades:**
 - Armazenamento **contíguo** na memória.
 - Acesso direto via **índice**: $O(1)$.
- **Limitação:** Tamanho fixo após a criação.

Endereço Base



Índice 0

Índice N-1

Analogia com a Memória

A memória RAM pode ser vista como um gigantesco array onde o “índice” é o endereço físico. O acesso a $a[i]$ traduz-se em apenas 3–4 instruções de CPU.

Objetivo: Encontrar primos até N .

1. Array $a[N]$ iniciado como `true`.
2. Para cada $i \geq 2$, se $a[i]$ é primo, marca múltiplos como `false`.

Por que Array?

O acesso direto $a[k]$ em $O(1)$ permite saltar entre múltiplos eficientemente. Complexidade: $O(N \log \log N)$.

```
1  class Primes {
2      public static void main(String[] args) {
3          int N = Integer.parseInt(args[0]);
4          boolean[] a = new boolean[N];
5
6          for (int i = 2; i < N; i++) a[i] = true;
7
8          for (int i = 2; i*i < N; i++) {
9              if (a[i]) {
10                 for (int j = i*i; j < N; j += i)
11                     a[j] = false;
12             }
13         }
14
15         for (int i = 2; i < N; i++)
16             if (a[i])
17                 System.out.print(i + " ");
18         System.out.println();
19     }
20 }
21
```

Limitação

O tamanho de um array é fixo na criação. Se ele encher, não há como adicionar mais elementos.

- **Solução:** O **Array Dinâmico** (como o `ArrayList` do Java) cresce automaticamente conforme necessário.
- **Funcionamento:**
 1. Aloca um array interno com uma capacidade inicial.
 2. Quando a capacidade é atingida, cria um novo array **maior**.
 3. Copia os elementos do antigo para o novo.

Antigo:



CHEIO! Novo:



espaço livre

```
1 public class MeuArrayDinamico {
2     private int[] data = new int[2];
3     private int size = 0;
4
5     public void add(int value) {
6         if (size == data.length) {
7             resize();
8         }
9         data[size++] = value;
10    }
11
12    private void resize() {
13        int[] newData =
14            new int[data.length * 2];
15        for (int i = 0; i < data.length; i++)
16            newData[i] = data[i];
17        data = newData;
18    }
19 }
20
```

Custo de Inserção no Fim

- **Caso comum:** $O(1)$ — apenas insere.
- **Caso de expansão:** $O(n)$ — copia tudo.
- **Amortizado:** $O(1)$.

Dica de Performance

Se você sabe o tamanho esperado, inicialize com essa capacidade:

```
ArrayList<String> l = new
    ArravList<>(10000);
```

Listas Simplesmente Ligadas

Motivação

Arrays são ótimos para **acesso por índice**, mas péssimos para **inserção e remoção no meio** — exigem deslocamento de todos os elementos subsequentes ($O(n)$).

- **Lista ligada:** conjunto de **nós**, cada um contendo um dado e um **link** para o próximo nó.
- Inserção e remoção em posição conhecida: $O(1)$.
- **Trade-off:** perde-se o acesso aleatório.

Operação	Array	Lista Ligada
Acesso por índice	$O(1)$	$O(n)$
Inserção/Remoção (posição conhecida)	$O(n)$	$O(1)$
Uso de memória	Contíguo	+1 ponteiro/nó

Definição

```
1  class Node {  
2      Object item;  
3      Node next;  
4  
5      Node(Object v) {  
6          item = v;  
7          next = null;  
8      }  
9  }  
10
```

Componentes do Nó:

- item: armazena o dado.
- next: referência para o próximo Node.

Acesso:

- x.item — o dado do nó x.
- x.next — o próximo nó.
- x.next.item — dado do seguinte.

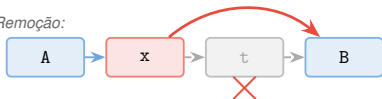
Cuidado

Sempre verifique se x e x.next não são null antes do acesso, para evitar NullPointerException.

Remoção após x

```
1  t = x.next;  
2  x.next = t.next;  
3
```

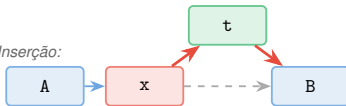
Remoção:



Inserção de t após x

```
1  t.next = x.next;  
2  x.next = t;  
3
```

Inserção:



Atenção à Ordem!

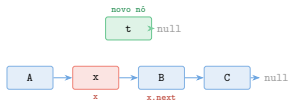
Na inserção, faça `t.next = x.next` **antes** de `x.next = t`. Inverter a ordem perde a referência ao restante da lista!

Código

```
1 t.next = x.next;  
2 x.next = t;  
3
```

Inserir o nó **t** **após** **x**.

Estado Inicial:



Código

```
1 t.next = x.next;  
2 x.next = t;  
3
```

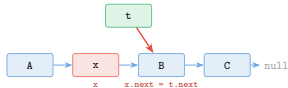
Inserir o nó **t** **após** **x**.

Estado Inicial:



Passo 1: `t.next = x.next`

(*t* aponta para o sucessor de *x*)



Código

```
1 t.next = x.next;  
2 x.next = t;  
3
```

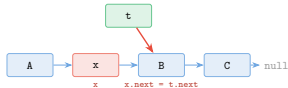
Inserir o nó **t** **após** **x**.

Estado Inicial:



Passo 1: `t.next = x.next`

(t aponta para o sucessor de x)

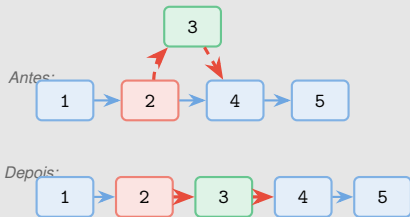


Passo 2: `x.next = t`

(x agora aponta para t)

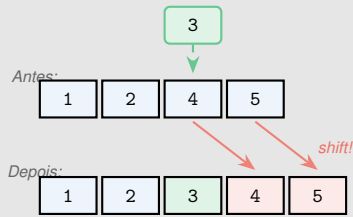


Lista Ligada — $O(1)$



Apenas 2 ponteiros alterados!

Array — $O(n)$



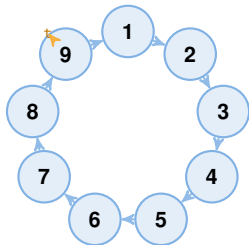
Todos os elementos após devem se mover!

- **Acesso por índice (k -ésimo item):**
 - **Arrays:** Acesso direto via $a[k]$ — tempo constante.
 - **Listas:** Requer travessia de k links — tempo linear.
- **Operação “não natural”:** Encontrar o item *anterior* a um dado nó exige percorrer do início ($O(n)$).
- **Garbage Collection:** Em Java, ao remover um nó, o sistema automaticamente recupera a memória. Em C/C++, seria necessário liberar manualmente.

Lição

A simplicidade da inserção/remoção é a razão de ser das listas; a dificuldade de acesso aleatório é seu principal compromisso.

- **Cenário:** N pessoas formam um círculo.
- **Regra:** Elimina-se cada M -ésima pessoa, fechando o círculo após cada saída.
- **Objetivo:** Determinar quem será o último sobrevivente.



Exemplo ($N = 9, M = 5$)

Eliminações: 5, 1, 7, 4, 3, 6, 9, 2.

Líder eleito: 8.

Por que Lista Ligada Circular?

```
1 class Josephus {
2     public static void main(String[] args){
3         int N = Integer.parseInt(args[0]);
4         int M = Integer.parseInt(args[1]);
5         Node t = new Node(1);
6         t.next = t; // lista circular
7         for (int i = 2; i <= N; i++) {
8             t.next = new Node(i, t.next);
9             t = t.next;
10        }
11        while (t != t.next) {
12            for (int i = 1; i < M; i++)
13                t = t.next; // pula M-1
14            Out.print(t.next.item + " ");
15            t.next = t.next.next; // remove
16        }
17        Out.println("\nLider: " + t.item);
18    }
19 }
20
```

Construção da lista circular:

1. Cria nó 1 apontando para si mesmo.
2. Insere 2, 3, ..., N logo após t , avançando t .

Eliminação:

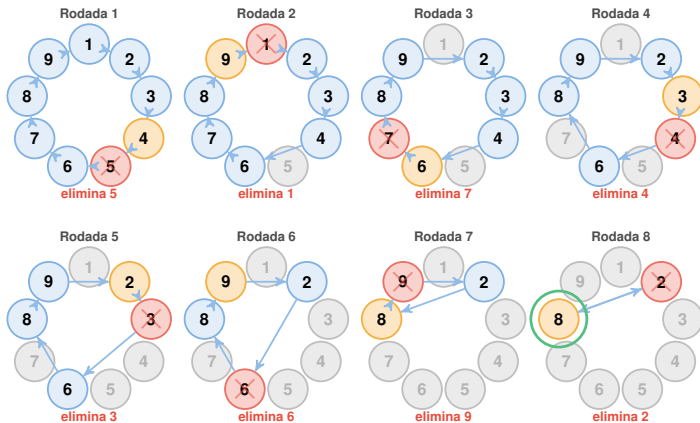
1. Avança t por $M-1$ posições.
2. Remove $t.next$ (o M -ésimo).
3. Repete até restar um nó.

Complexidade

$O(N \cdot M)$ no total.

Cada rodada percorre M nós

Josephus: Processo de Eliminação ($N = 9$, $M = 5$)



Ordem: 5, 1, 7, 4, 3, 6, 9, 2

Líder: 8

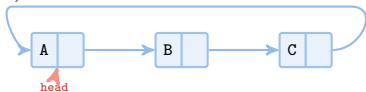
Algoritmo	Estrutura Ideal	Motivo
Crivo de Eratóstenes	Array	Acesso aleatório rápido via índice.
Problema de Josephus	Lista Circular	Remoção eficiente de itens.

Lição Fundamental

O uso de uma lista para o Crivo seria caro (falta de acesso direto). O uso de um array para Josephus seria caro (deslocamento na remoção). O design eficiente nasce da **interação** entre estrutura e algoritmo.

Convenções de Organização de Listas

1. Circular, nunca vazia



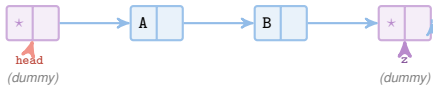
2. Ref. head, cauda null



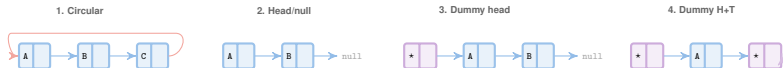
3. Nó sentinela head, cauda null



4. Sentinelas head e tail



Propriedade	1. Circular	2. Head/Null	3. Dummy Head	4. Dummy H+T
Lista vazia?	Não	Sim	Sim	Sim
Caso especial na inserção	Não	Sim	Não	Não
Testa null na travessia	Não	Sim	Sim	Não
Memória extra	Nenhuma	Nenhuma	+1 nó	+2 nós
Risco de NullPtrExc	Não	Sim	Sim	Não
Uso típico	Josephus, round-robin	Uso geral, pilhas/filas	Código uniforme	Busca sequencial



Listas Duplamente Ligadas

Cada nó mantém **dois** ponteiros:

- prev — nó anterior
- next — nó seguinte



Permite percorrer a lista em **ambas as direções** e remover um nó conhecendo **apenas** o seu ponteiro.

Invariante fundamental:

```
x == x.next.prev == x.prev.next
```



Código

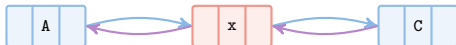
```
1 // Remover o nó x
2 x.prev.next = x.next;
3 x.next.prev = x.prev;
4
```

Basta conhecer x para removê-lo!
Nenhuma busca pelo predecessor.

Diferença crucial:

Na lista simples, precisamos do nó *anterior* a x . Na dupla, x já contém $x.prev$.

Antes:



Código

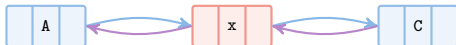
```
1 // Remover o nó x
2 x.prev.next = x.next;
3 x.next.prev = x.prev;
4
```

Basta conhecer x para removê-lo!
Nenhuma busca pelo predecessor.

Diferença crucial:

Na lista simples, precisamos do nó *anterior* a x . Na dupla, x já contém $x.prev$.

Antes:



Inserir t após x

```
1  t.next = x.next;  
2  t.prev = x;  
3  x.next.prev = t;  
4  x.next = t;  
5
```

Inserir t antes de x

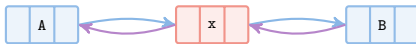
```
1  t.prev = x.prev;  
2  t.next = x;  
3  x.prev.next = t;  
4  x.prev = t;  
5
```

Inserir t após x :

novο



Antes:



Operação	Lista Simples	Lista Dupla
Inserir após x	$O(1)$ — 2 ponteiros	$O(1)$ — 4 ponteiros
Inserir antes de x	$O(n)$ — buscar predecessor	$O(1)$ — 4 ponteiros
Remover x (dado x)	$O(n)$ — buscar predecessor	$O(1)$ — 2 ponteiros
Remover após x	$O(1)$ — 1 ponteiro	$O(1)$ — 2 ponteiros
Espaço por nó	1 ponteiro	2 ponteiros

Quando usar lista simples:

Inserção/remoção sempre no início ou após um nó conhecido. Economia de memória.

Quando usar lista dupla:

Remoção direta de nó, travessia bidirecional, nós compartilhados entre estruturas (ex: LRU cache).

Recapitulação

Operação	Array Estático	Array Dinâmico	Lista Ligada
Acesso por índice	$O(1)$	$O(1)$	$O(n)$
Inserção no fim	—	$O(1)$ amortizado	$O(1)$
Inserção no meio	$O(n)$	$O(n)$	$O(1)^*$
Remoção (posição conhecida)	$O(n)$	$O(n)$	$O(1)^*$
Memória	Contígua, fixa	Contígua, cresce	Fragmentada

* Se já temos a referência ao nó.

Mensagem Central

Não existe estrutura “melhor” em absoluto. A escolha depende do **padrão de acesso** do algoritmo que a utiliza.

- **Próxima aula:** Pilhas e Filas — ADTs construídos sobre arrays e listas.
- **Exercícios sugeridos:**
 1. Implemente um `ArrayList` simplificado com operação `remove(int index)`.
 2. Resolva o problema de Josephus para diferentes valores de N e M .
 3. Implemente uma lista duplamente ligada com sentinelas e operações de inserção/remoção.
- **Leitura:** Sedgewick, capítulos sobre estruturas elementares.